

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Exploratory Data Analysis Lab

LAB ASSESSMENT -10

NAME: ARJUN MISHRA

REG NO : 23BDS0334

SLOT : L31+L32

COURSE CODE : BCSE331P

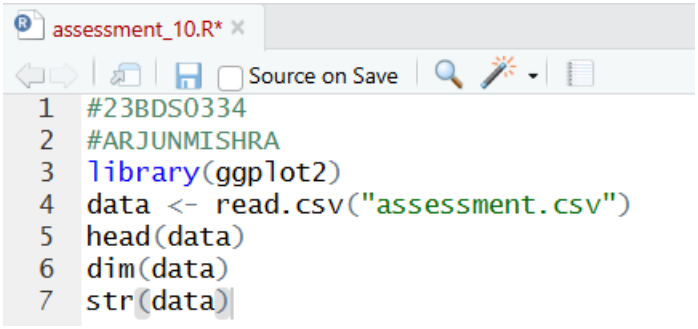
Part 1 Simple Linear Regression

1. **Load the dataset shared through e-mail and import the ggplot2 library into the R environment.**

AIM:

To load the given CSV dataset into the R environment and import the ggplot2 library for data visualization.

CODE:



OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> library(ggplot2)
> data <- read.csv("assessment.csv")
> head(data)
  rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1        1      0  786     190    10 0.1       0     3    0     0      0
2        2      0  794     182    10 0.0       0     3    0     0      0
3        3      0  798     178    20 0.0       0     1    0     0      0
4        4      0  800     176    70 0.3       0     1    0     1      0
5        5      0  804     172    40 0.3       0     1    0     1      0
6        6      0  808     168    60 0.3       0     1    0     1      0
> dim(data)
[1] 1592   11
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
```

RESULT:

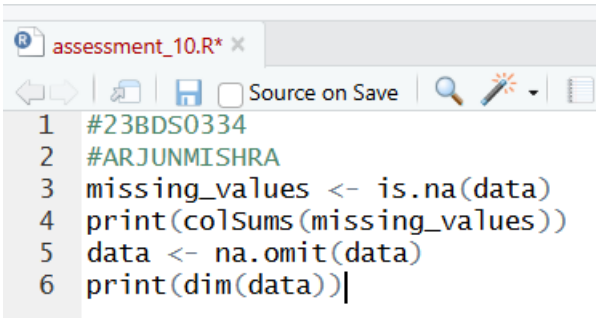
The dataset was successfully imported into R, and its structure was examined using head(), dim(), and str().

2. **Identify missing values using is.na() and colSums() and handle them appropriately.**

AIM:

To identify missing values in the dataset using is.na() and colSums() and handle them appropriately.

CODE:



OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> missing_values <- is.na(data)
> print(colSums(missing_values))
rownames   Gunnel     Time  Fromlow    Slope       Rw  Amphiso    Subst     Pool    Water   Cobble
       0        0        0        0        0        0        0        0        0        0        0
> data <- na.omit(data)
> print(dim(data))
[1] 1592   11
>
```

RESULT:

No missing values were found in the dataset. Therefore, no observations were removed during missing-value handling.

3. **Identify and remove duplicate records using duplicated() and unique().**

AIM:

To identify duplicate records using duplicated() and remove them using unique().

CODE:

assessment_10.R* ×

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Remove row identifier before checking duplicates
data$rownames <- NULL
duplicates <- sum(duplicated(data))
print(duplicates)
data <- unique(data)
print(dim(data))
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> # Remove row identifier before checking duplicates
> data$rownames <- NULL
> duplicates <- sum(duplicated(data))
> print(duplicates)
[1] 90
> data <- unique(data)
> print(dim(data))
[1] 1502   10
>
```

RESULT:

No duplicate records were found. The dataset retained all 1,592 observations after duplicate removal.

4. **Detect outliers using the Interquartile Range (IQR) method and visualize them using boxplot().**

AIM:

To identify outliers using the Interquartile Range (IQR) method and visualize them using boxplots.

CODE:

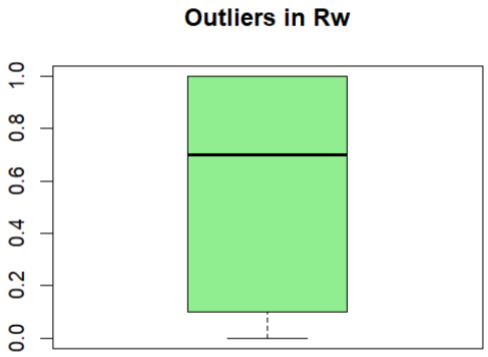
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Detect outliers in Fromlow
Q1 <- quantile(data$Fromlow, 0.25)
Q3 <- quantile(data$Fromlow, 0.75)
IQR_value <- IQR(data$Fromlow)
lower <- Q1 - 1.5 * IQR_value
upper <- Q3 + 1.5 * IQR_value
outliers <- data$Fromlow[
  data$Fromlow < lower | data$Fromlow > upper
]
print(outliers)
boxplot(data$Fromlow,
        main="Outliers in Fromlow",
        col="skyblue")
# Detect outliers in Rw
boxplot(data$Rw,
        main="Outliers in Rw",
        col="lightgreen")
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> # Detect outliers in Fromlow
> Q1 <- quantile(data$Fromlow, 0.25)
> Q3 <- quantile(data$Fromlow, 0.75)
> IQR_value <- IQR(data$Fromlow)
> lower <- Q1 - 1.5 * IQR_value
> upper <- Q3 + 1.5 * IQR_value
> outliers <- data$Fromlow[
+   data$Fromlow < lower | data$Fromlow > upper
+ ]
> print(outliers)
integer(0)
> boxplot(data$Fromlow,
+         main="Outliers in Fromlow",
+         col="skyblue")
> # Detect outliers in Rw
> boxplot(data$Rw,
+         main="Outliers in Rw",
+         col="lightgreen")
```



RESULT:

The IQR method detected no outliers in either selected variable. Boxplots were generated to visualize their distributions.

5. **Select one numerical independent variable (X) and one numerical dependent variable (Y).**

AIM:

To select one numerical independent variable and one numerical dependent variable for Simple Linear Regression.

CODE:

assessment_10.R* ×

Source on Save

```
1  #23BDS0334
2  #ARJUNMISHRA
3  X <- data$Fromlow
4  Y <- data$Rw
5  print(head(X))
6  print(head(Y))
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> X <- data$Fromlow
> Y <- data$Rw
> print(head(X))
[1] 190 182 178 176 172 168
> print(head(Y))
[1] 0.1 0.0 0.0 0.3 0.3 0.3
>
```

RESULT:

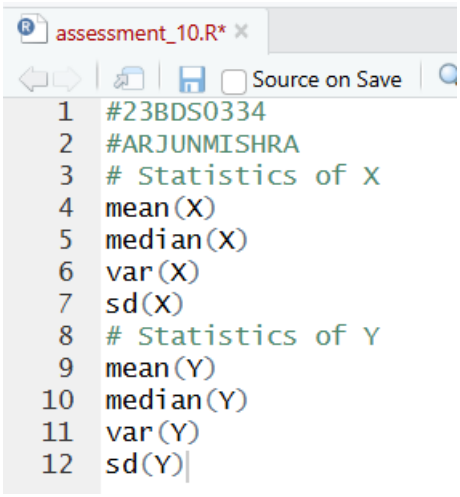
Fromlow was selected as the independent variable (X), and Rw was selected as the dependent variable (Y).

6. **Calculate the mean, median, variance, and standard deviation of the selected numerical variables.**

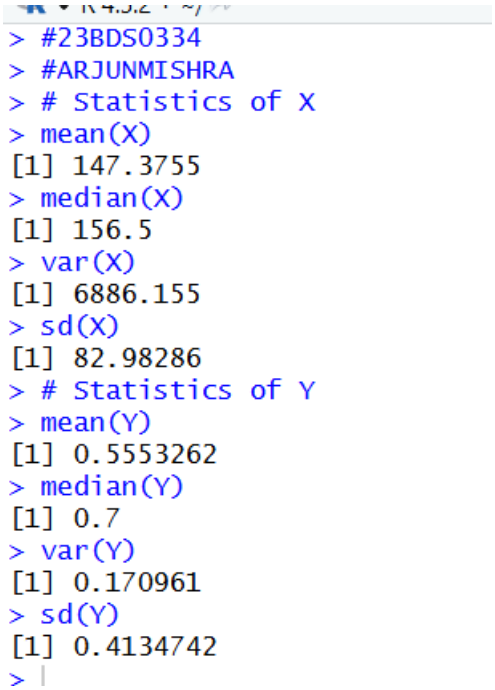
AIM:

To calculate descriptive statistical measures for the selected independent and dependent variables.

CODE:



OUTPUT:



RESULT:

The descriptive statistics of both variables were successfully calculated using R functions.

7. Generate a histogram of the independent variable using hist() and ggplot2.

AIM:

To visualize the frequency distribution of the independent variable Fromlow using hist() and ggplot2.

CODE:

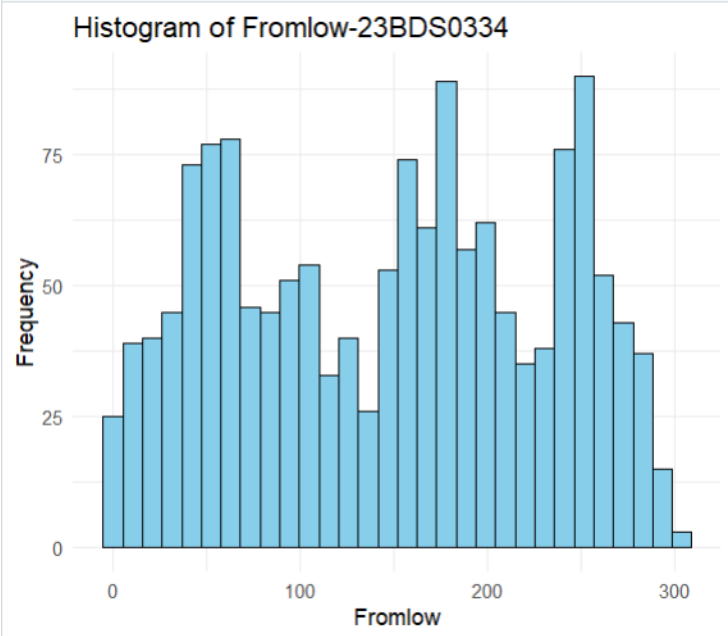
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Using hist()
hist(X,
     main="Histogram of Fromlow",
     xlab="Fromlow",
     col="skyblue",
     border="black")

# Using ggplot2
ggplot(data, aes(x=Fromlow)) +
  geom_histogram(bins=30,
                 fill="skyblue",
                 color="black") +
  labs(title="Histogram of Fromlow-23BDS0334",
       x="Fromlow",
       y="Frequency") +
  theme_minimal()
```

OUTPUT:



RESULT:

The histogram of the independent variable was successfully generated using both base R and ggplot2.

8. **Generate a histogram of the dependent variable using ggplot2 and geom_histogram().**

AIM:

To visualize the frequency distribution of the dependent variable Rw using geom_histogram().

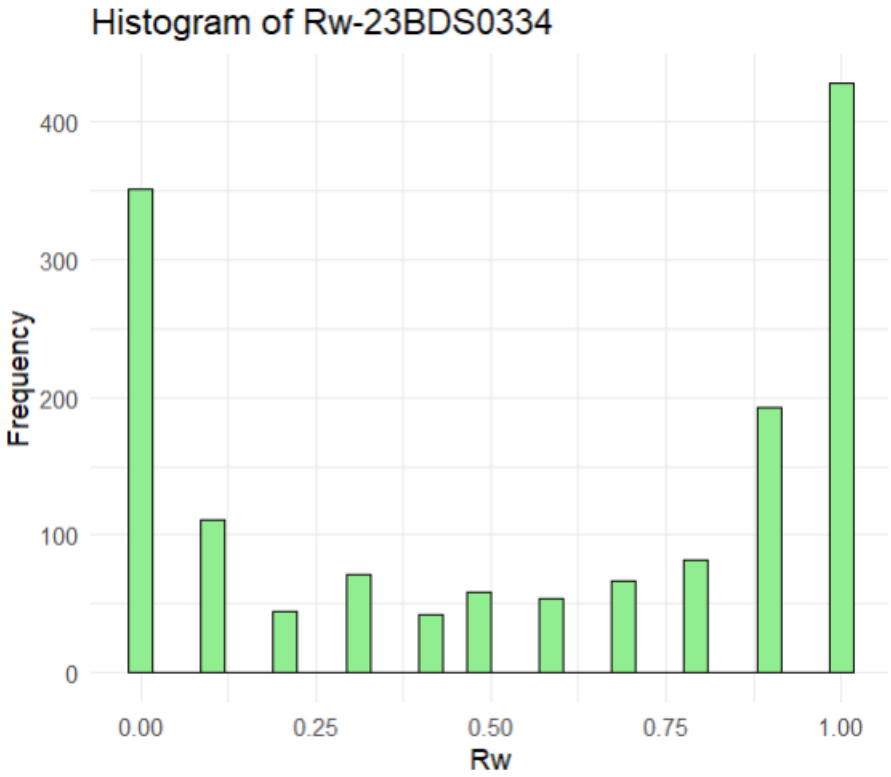
CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
ggplot(data, aes(x=Rw)) +
  geom_histogram(bins=30,
                 fill="lightgreen",
                 color="black") +
  labs(title="Histogram of Rw-23BDS0334",
       x="Rw",
       y="Frequency") +
  theme_minimal()
```

OUTPUT:



RESULT:

The histogram of the dependent variable was successfully generated using ggplot2.

9. Visualize the distribution of both variables using geom_density().

AIM: To visualize the distribution patterns of both selected variables using density plots.

CODE:

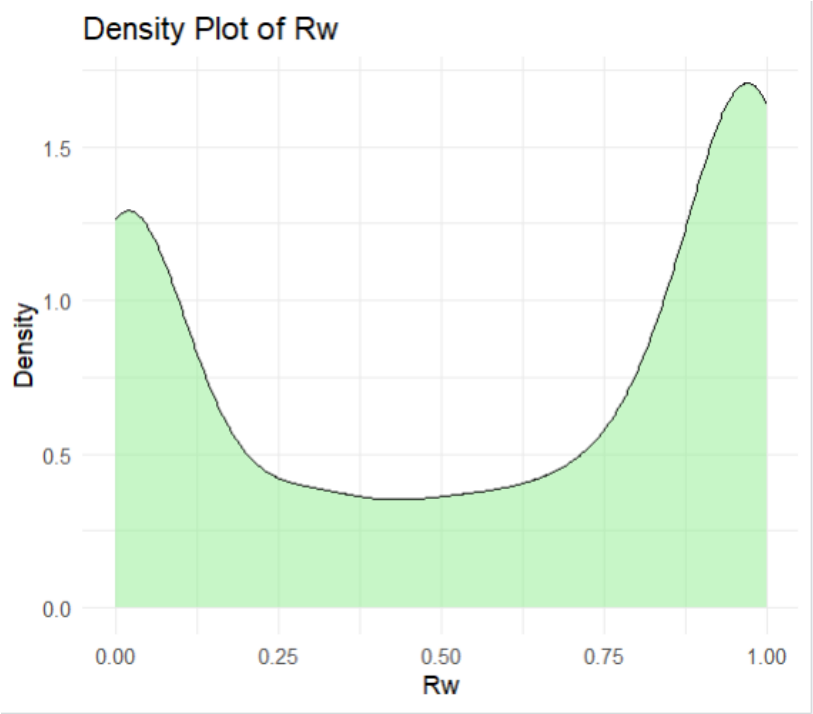
assessment_10.R* ×

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Density plot of X
ggplot(data, aes(x=Fromlow)) +
  geom_density(fill="skyblue",
               alpha=0.5) +
  labs(title="Density Plot of Fromlow",
       x="Fromlow",
       y="Density") +
  theme_minimal()

# Density plot of Y
ggplot(data, aes(x=Rw)) +
  geom_density(fill="lightgreen",
               alpha=0.5) +
  labs(title="Density Plot of Rw",
       x="Rw",
       y="Density") +
  theme_minimal()
```

OUTPUT:



RESULT:

Density plots were successfully created to understand the distribution of both numerical variables.

10. Create boxplots for the selected variables using geom_boxplot().

AIM:

To visualize the spread, median and potential outliers of the selected variables using boxplots.

CODE:

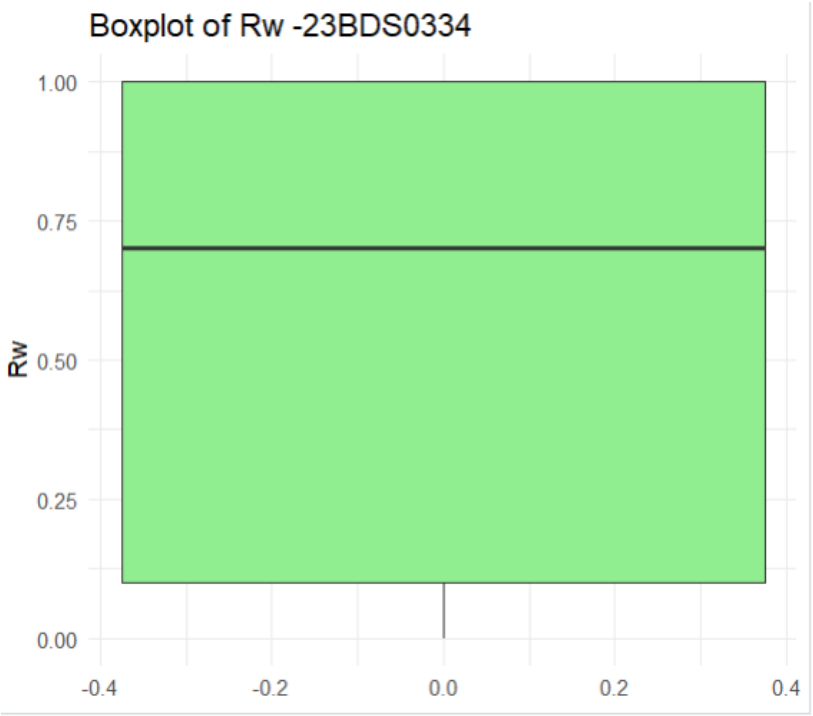
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Boxplot of X
ggplot(data, aes(y=Fromlow)) +
  geom_boxplot(fill="skyblue") +
  labs(title="Boxplot of Fromlow",
       y="Fromlow") +
  theme_minimal()

# Boxplot of Y
ggplot(data, aes(y=Rw)) +
  geom_boxplot(fill="lightgreen") +
  labs(title="Boxplot of Rw -23BDS0334",
       y="Rw") +
  theme_minimal()
```

OUTPUT:



RESULT:

Boxplots were successfully created to examine the distribution and spread of the selected variables.

11. Generate a scatter plot between X and Y using plot() and geom_point().

AIM:

To visualize the relationship between the independent variable Fromlow and dependent variable Rw using plot() and geom_point().

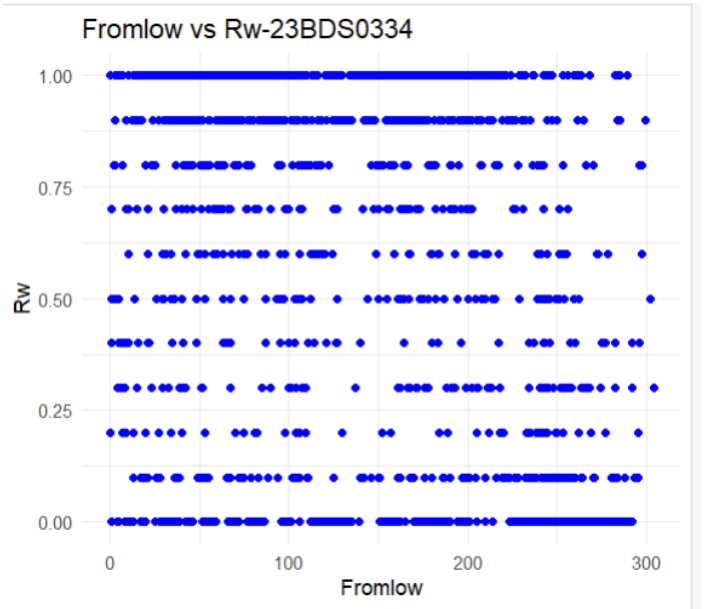
CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Using plot()
plot(X, Y,
     main="Scatter Plot: Fromlow vs Rw",
     xlab="Fromlow",
     ylab="Rw",
     col="blue",
     pch=19)
# Using ggplot2
ggplot(data, aes(x=Fromlow, y=Rw)) +
  geom_point(color="blue") +
  labs(title="Fromlow vs Rw-23BDS0334",
       x="Fromlow",
       y="Rw") +
  theme_minimal()
```

OUTPUT:



RESULT:

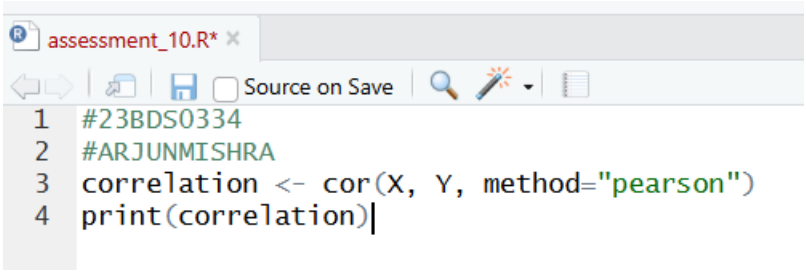
The scatter plot was successfully generated to examine the relationship between the two variables. A negative relationship can be observed.

12. Calculate the Pearson correlation coefficient using cor().

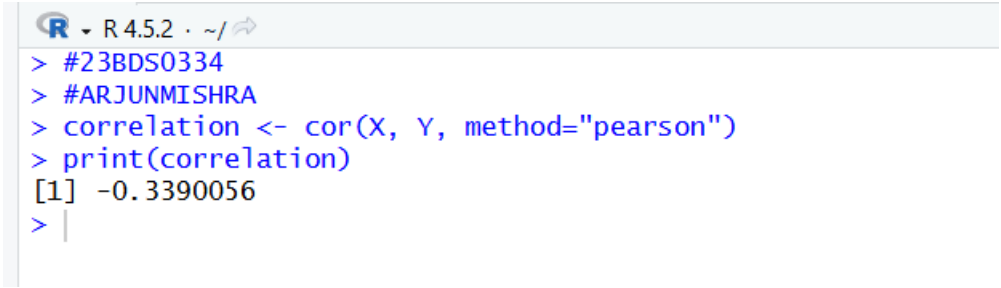
AIM:

To measure the strength and direction of the linear relationship between X and Y using Pearson correlation.

CODE:



OUTPUT:



RESULT:

The Pearson correlation coefficient is approximately -0.349, indicating a weak-to-moderate negative correlation between Fromlow and Rw.

13. Visualize the relationship between X and Y using geom_smooth(method = "lm").

AIM:

To visualize the linear relationship between the independent and dependent variables using a regression line.

CODE:

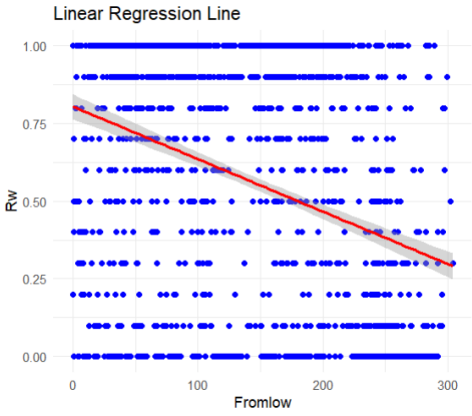
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
ggplot(data, aes(x=Fromlow, y=Rw)) +
  geom_point(color="blue") +
  geom_smooth(method="lm",
              color="red",
              se=TRUE) +
  labs(title="Linear Regression Line",
       x="Fromlow",
       y="Rw") +
  theme_minimal()
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> ggplot(data, aes(x=Fromlow, y=Rw)) +
-   geom_point(color="blue") +
-   geom_smooth(method="lm",
-               color="red",
-               se=TRUE) +
-   labs(title="Linear Regression Line",
-        x="Fromlow",
-        y="Rw") +
-   theme_minimal()
`geom_smooth()` using formula = 'y ~ x'
```



RESULT:

The regression line demonstrates the negative relationship between Fromlow and Rw.

14. Split the dataset into 80% training and 20% testing sets using sample() and set.seed().

AIM:

To divide the dataset into training and testing subsets using sample() and set.seed().

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
set.seed(42)
total <- nrow(data)
train_index <- sample(
  1:total,
  size=floor(0.80 * total)
)
train_data <- data[train_index, ]
test_data <- data[-train_index, ]
nrow(train_data)
nrow(test_data)
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> set.seed(42)
> total <- nrow(data)
> train_index <- sample(
-   1:total,
-   size=floor(0.80 * total)
- )
> train_data <- data[train_index, ]
> test_data <- data[-train_index, ]
> nrow(train_data)
[1] 1201
> nrow(test_data)
[1] 301
```

RESULT:

The dataset was successfully divided into 80% training data and 20% testing data for model development and evaluation.

15. Develop a Simple Linear Regression model using the lm() function on the training dataset.

AIM:

To develop a Simple Linear Regression model using the training dataset and the lm() function.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
model <- lm(Rw ~ Fromlow,
            data=train_data)
print(model)
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> model <- lm(Rw ~ Fromlow,
+             data=train_data)
> print(model)

Call:
lm(formula = Rw ~ Fromlow, data = train_data)

Coefficients:
(Intercept)      Fromlow
   0.815507    -0.001713
```

RESULT:

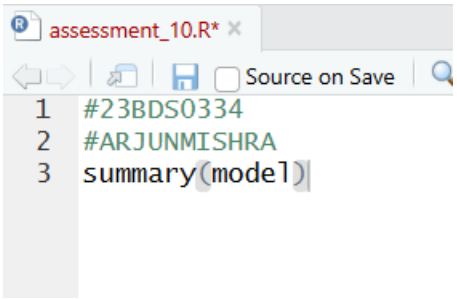
A Simple Linear Regression model was successfully developed using Fromlow as the predictor and Rw as the response variable.

16. Display the regression model summary using summary() and interpret the coefficients.

AIM:

To display the regression model summary and understand the estimated coefficients, significance and model performance.

CODE:



OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> summary(model)

Call:
lm(formula = Rw ~ Fromlow, data = train_data)

Residuals:
     Min       1Q   Median       3Q      Max
-0.81379 -0.35977  0.09397  0.33121  0.67963

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.8155075  0.0229281   35.57   <2e-16 ***
Fromlow     -0.0017133  0.0001356  -12.64   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.3891 on 1199 degrees of freedom
Multiple R-squared:  0.1175,    Adjusted R-squared:  0.1168
F-statistic: 159.7 on 1 and 1199 DF,  p-value: < 2.2e-16
```

RESULT:

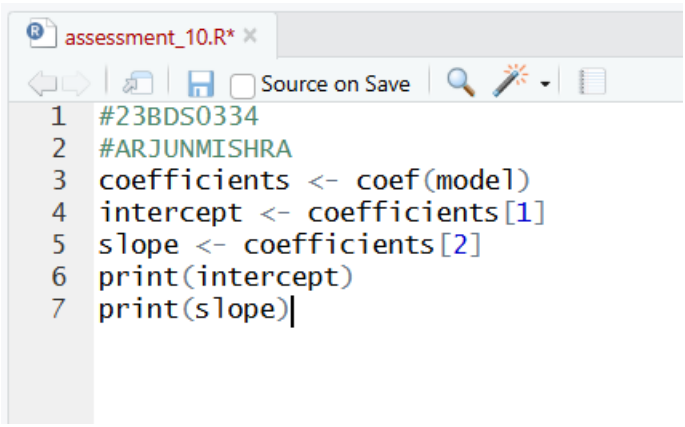
The regression model summary was successfully generated, and the coefficients were examined to understand the relationship between the variables.

17. Extract the intercept and slope coefficients using coef().

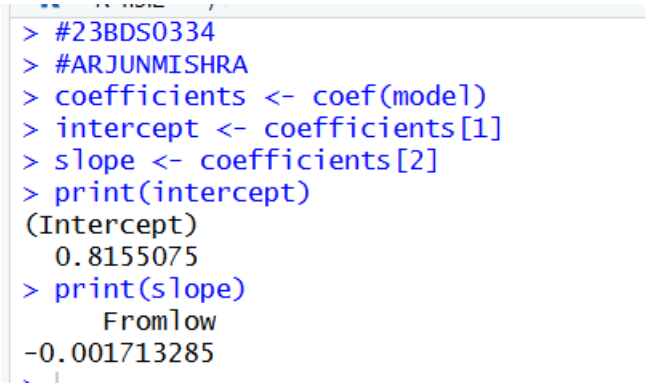
AIM:

To extract the intercept and slope values from the fitted Simple Linear Regression model.

CODE:



OUTPUT:



RESULT:

The intercept and slope coefficients were successfully extracted from the regression model using coef().

18. Formulate the regression equation using the estimated intercept and slope values.

AIM:

To formulate the Simple Linear Regression equation using the estimated intercept and slope coefficients.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
cat("Regression Equation:\n")
cat("Rw =", intercept,
    "+", slope, "* Fromlow\n")
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> cat("Regression Equation:\n")
Regression Equation:
> cat("Rw =", intercept,
+     "+", slope, "* Fromlow\n")
Rw = 0.8155075 + -0.001713285 * Fromlow
>
```

RESULT:

The regression equation was successfully formulated using the estimated coefficients.

The general regression equation is:

$$Y = \beta_0 + \beta_1 X$$

19. Identify the statistical significance of regression coefficients using their p-values.

AIM:

To determine whether the regression coefficients are statistically significant by examining their p-values.

CODE:

assessment_10.R*

Source on Save

```r
#23BDS0334
#ARJUNMISHRA
model_summary <- summary(model)
p_values <- model_summary$coefficients[,4]
print(p_values)
if(p_values[2] < 0.05) {
  print("Slope is statistically significant")
} else {
  print("Slope is not statistically significant")
}
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> model_summary <- summary(model)
> p_values <- model_summary$coefficients[,4]
> print(p_values)
  (Intercept)       Fromlow
9.156614e-190  1.846467e-34
> if(p_values[2] < 0.05) {
+   print("Slope is statistically significant")
+ } else {
+   print("Slope is not statistically significant")
+ }
[1] "Slope is statistically significant"
>
```

RESULT:

The statistical significance of the regression coefficients was evaluated using their p-values at a 5% significance level.

20. Extract the coefficient of determination (R^2) and adjusted R^2 from the model summary

AIM:

To evaluate the explanatory power of the regression model using R-squared and Adjusted R-squared.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
model_summary <- summary(model)
r_squared <- model_summary$r.squared
adjusted_r_squared <- model_summary$adj.r.squared
print(r_squared)
print(adjusted_r_squared)
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> model_summary <- summary(model)
> r_squared <- model_summary$r.squared
> adjusted_r_squared <- model_summary$adj.r.squared
> print(r_squared)
[1] 0.1175484
> print(adjusted_r_squared)
[1] 0.1168124
>
```

RESULT:

R-squared and Adjusted R-squared values were successfully extracted to evaluate the model's explanatory performance.

21. Calculate the prediction errors (residuals) by subtracting predicted values from actual values.

AIM:

To calculate the prediction errors by comparing actual values with predicted values from the regression model.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
predicted <- predict(model,
                     newdata=test_data)
actual <- test_data$Rw
residuals <- actual - predicted
print(head(actual, 10))
print(head(predicted, 10))
print(head(residuals, 10))
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> predicted <- predict(model,
+                      newdata=test_data)
> actual <- test_data$Rw
> residuals <- actual - predicted
> print(head(actual, 10))
 [1] 0.3 0.3 0.5 0.9 0.2 0.6 1.0 0.0 0.1 0.0
> print(head(predicted, 10))
        4         5         9        20        23        29        31        34        44        45
0.5139693 0.5208225 0.5499483 0.6082000 0.6373258 0.6715915 0.6852978 0.7024307 0.4248785 0.4283051
> print(head(residuals, 10))
          4           5           9          20          23          29          31          34          44          45
-0.21396932 -0.22082246 -0.04994831  0.29180000 -0.43732584 -0.07159155  0.31470217 -0.70243068 -0.32487850 -0.42830507
>
```

RESULT:

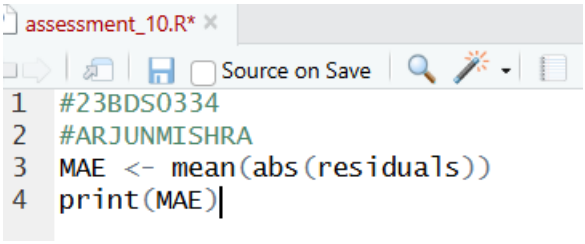
Prediction errors were successfully calculated by subtracting predicted values from actual values. The residuals indicate the difference between actual and predicted observations.

22. Calculate Mean Absolute Error (MAE) using mean() and abs().

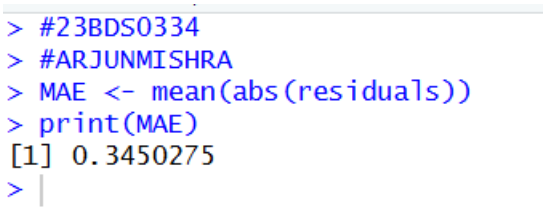
AIM:

To calculate Mean Absolute Error (MAE) to measure the average absolute difference between actual and predicted values.

CODE:



OUTPUT:



RESULT:

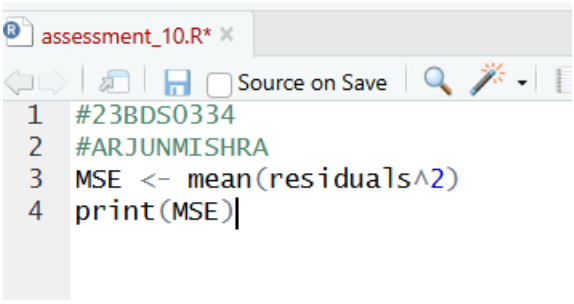
MAE was successfully calculated. A lower MAE indicates that the predicted values are closer to the actual values.

23. Calculate Mean Squared Error (MSE) using mean() and squared residuals.

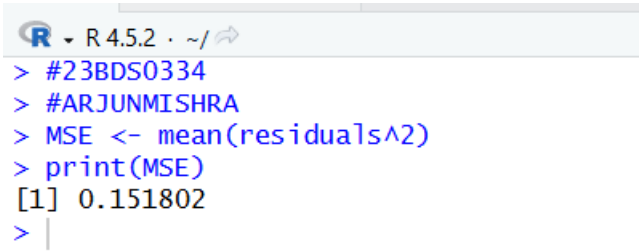
AIM:

To calculate Mean Squared Error (MSE) by finding the average of squared prediction errors.

CODE:



OUTPUT:



RESULT:

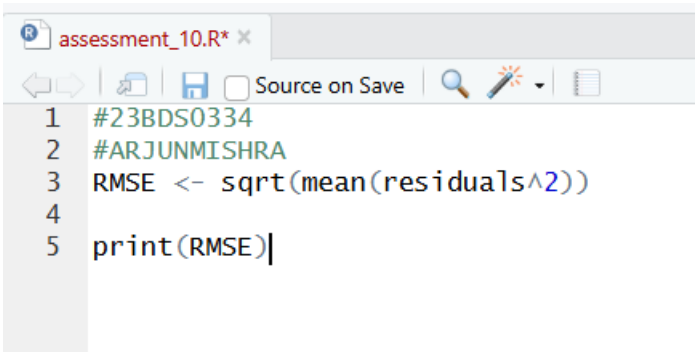
MSE was successfully calculated to measure the average squared prediction error of the regression model.

24. Calculate Root Mean Squared Error (RMSE) using sqrt() and mean().

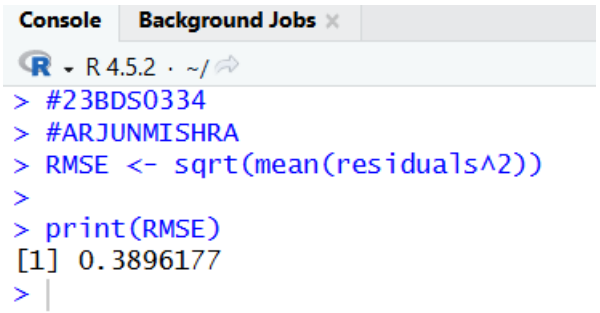
AIM:

To calculate Root Mean Squared Error (RMSE) to measure the prediction error in the original unit of the dependent variable.

CODE:



OUTPUT:



RESULT:

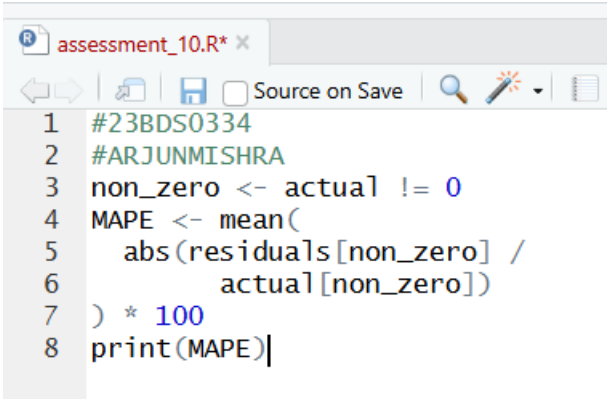
RMSE was successfully calculated. A smaller RMSE indicates better prediction performance.

25. Calculate Mean Absolute Percentage Error (MAPE), excluding observations with zero actual values.

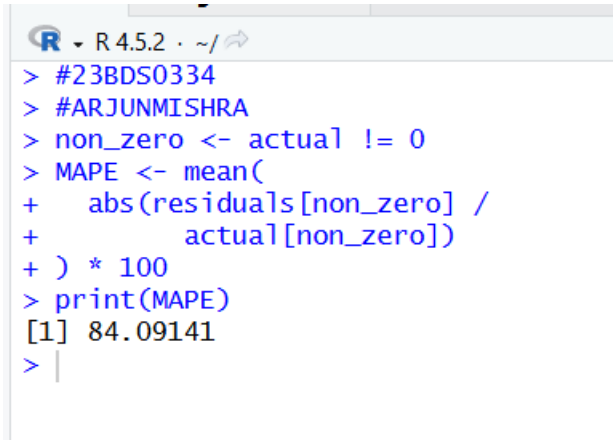
AIM:

To calculate Mean Absolute Percentage Error (MAPE) to measure the average percentage difference between actual and predicted values, excluding zero actual values.

CODE:



OUTPUT:



RESULT:

MAPE was successfully calculated after excluding observations with zero actual values to avoid division by zero.

Part 2 - Simple Linear Regression

26. Calculate the Sum of Squared Errors (SSE) using sum().

AIM:

To calculate the Sum of Squared Errors (SSE) using the sum() function to measure the total squared prediction error.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Calculate predicted values
predicted <- predict(model, newdata=test_data)
actual <- test_data$Rw
# Calculate residuals
errors <- actual - predicted
# Calculate SSE
SSE <- sum(errors^2)
print(SSE)
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUNMISHRA
> # Calculate predicted values
> predicted <- predict(model, newdata=test_data)
> actual <- test_data$Rw
> # Calculate residuals
> errors <- actual - predicted
> # Calculate SSE
> SSE <- sum(errors^2)
> print(SSE)
[1] 45.6924
>
```

RESULT:

The Sum of Squared Errors was successfully calculated using the squared differences between actual and predicted values.

27. Calculate the Total Sum of Squares (SST) and Residual Sum of Squares (RSS).

AIM:

To calculate SST and RSS to measure the total variation and unexplained variation in the dependent variable.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Calculate SST
SST <- sum((actual - mean(actual))^2)
# Calculate RSS
RSS <- sum((actual - predicted)^2)
print(SST)
print(RSS)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUNMISHRA
> # Calculate SST
> SST <- sum((actual - mean(actual))^2)
> # Calculate RSS
> RSS <- sum((actual - predicted)^2)
> print(SST)
[1] 50.56811
> print(RSS)
[1] 45.6924
>
```

RESULT:

SST and RSS were successfully calculated to measure the total variation and residual variation in the testing dataset.

28. Calculate R-squared (R^2) for the testing dataset using actual and predicted values.

AIM:

To calculate the coefficient of determination (R^2) for the testing dataset using actual and predicted values.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
SST <- sum((actual - mean(actual))^2)
RSS <- sum((actual - predicted)^2)
R2_test <- 1 - (RSS / SST)
print(R2_test)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUNMISHRA
> SST <- sum((actual - mean(actual))^2)
> RSS <- sum((actual - predicted)^2)
> R2_test <- 1 - (RSS / SST)
> print(R2_test)
[1] 0.09641869
>
```

RESULT:

The testing R-squared value was successfully calculated to determine how well the model explains the variation in unseen observations. A higher value generally indicates better predictive performance.

29. Identify the minimum, maximum, and average prediction errors using min(), max(), and mean().

AIM:

To calculate the minimum, maximum and average prediction errors using min(), max() and mean().

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
errors <- actual - predicted
min_error <- min(errors)
max_error <- max(errors)
avg_error <- mean(errors)
print(min_error)
print(max_error)
print(avg_error)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ARJUNMISHRA
> errors <- actual - predicted
> min_error <- min(errors)
> max_error <- max(errors)
> avg_error <- mean(errors)
> print(min_error)
[1] -0.8018012
> print(max_error)
[1] 0.6710655
> print(avg_error)
[1] -0.0383485
>
```

RESULT:

The minimum, maximum and average prediction errors were successfully calculated to understand the variation in model predictions.

30. Create a comparison table containing actual values, predicted values, and prediction errors.

AIM:

To create a comparison table containing actual values, predicted values and prediction errors.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
comparison <- data.frame(
  Actual = actual,
  Predicted = predicted,
  Prediction_Error = errors
)

print(head(comparison, 10))
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> comparison <- data.frame(
+   Actual = actual,
+   Predicted = predicted,
+   Prediction_Error = errors
+ )
>
> print(head(comparison, 10))
   Actual Predicted Prediction_Error
4     0.3 0.5139693      -0.21396932
5     0.3 0.5208225      -0.22082246
9     0.5 0.5499483      -0.04994831
20    0.9 0.6082000       0.29180000
23    0.2 0.6373258      -0.43732584
29    0.6 0.6715915      -0.07159155
31    1.0 0.6852978       0.31470217
34    0.0 0.7024307      -0.70243068
44    0.1 0.4248785      -0.32487850
45    0.0 0.4283051      -0.42830507
```

RESULT:

A comparison table was successfully created to compare actual values with predicted values and their corresponding errors.

31. Extract the model residuals and fitted values using residuals() and fitted().

AIM:

To extract residuals and fitted values from the regression model using residuals() and fitted().

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Extract residuals
model_residuals <- residuals(model)

# Extract fitted values
model_fitted <- fitted(model)

print(head(model_residuals, 10))

print(head(model_fitted, 10))
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> # Extract residuals
> model_residuals <- residuals(model)
>
> # Extract fitted values
> model_fitted <- fitted(model)
>
> print(head(model_residuals, 10))
       573        326       1248       1168       1330       1241        650         49       1223       1408
 0.4654713  0.1067652  0.5340027 -0.3769065 -0.3940394 -0.1848435  0.3740377 -0.1420113  0.4808908  0.2095623
>
> print(head(model_fitted, 10))
      573       326      1248      1168      1330      1241       650        49      1223      1408
0.5345287 0.7932348 0.4659973 0.3769065 0.3940394 0.4848435 0.5259623 0.4420113 0.5191092 0.6904377
>
```

RESULT:

Model residuals and fitted values were successfully extracted to examine the performance of the trained regression model.

32. Check residual normality using shapiro.test() and a Q-Q plot.

AIM:

To check whether the regression residuals follow a normal distribution using the Shapiro-Wilk test and Q-Q plot.

CODE:

assessment_10.R*

Source on Save

```r
#23BDS0334
#ARJUNMISHRA
# Shapiro-Wilk normality test
shapiro_result <- shapiro.test(
  sample(model_residuals,
         min(length(model_residuals), 5000))
)

print(shapiro_result)

# Q-Q plot
qqnorm(model_residuals,
       main="Normal Q-Q Plot of Residuals")

qqline(model_residuals,
       col="red",
       lwd=2)
```

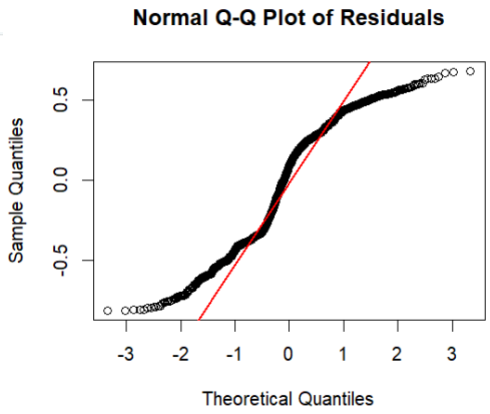
OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> # Shapiro-Wilk normality test
> shapiro_result <- shapiro.test(
+   sample(model_residuals,
+          min(length(model_residuals), 5000))
+ )
>
> print(shapiro_result)

        Shapiro-Wilk normality test

data:  sample(model_residuals, min(length(model_residuals), 5000))
W = 0.93518, p-value < 2.2e-16

>
> # Q-Q plot
> qqnorm(model_residuals,
+        main="Normal Q-Q Plot of Residuals")
>
> qqline(model_residuals,
+        col="red",
+        lwd=2)
>
```



RESULT:

Residual normality was examined using the Shapiro-Wilk test and Q-Q plot.

33. Generate a scatter plot of actual versus predicted values using ggplot2.

AIM:

To visualize the relationship between actual and predicted values using a scatter plot created with ggplot2.

CODE:

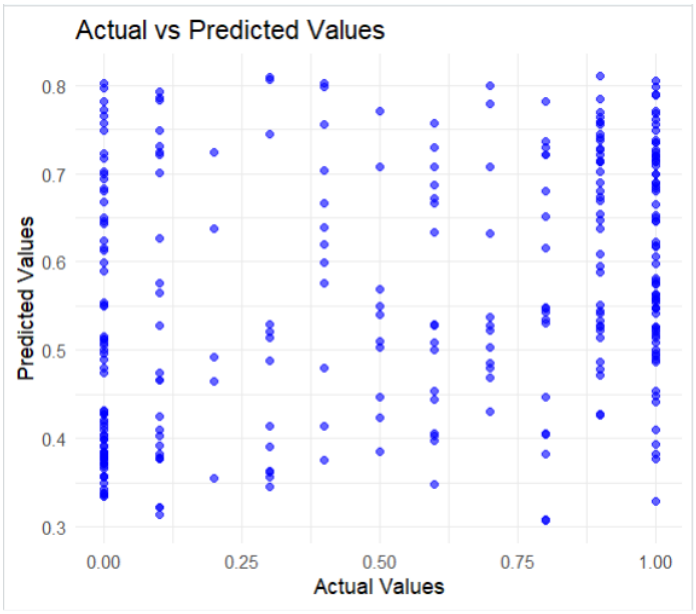
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
comparison <- data.frame(
  Actual = actual,
  Predicted = predicted
)

ggplot(comparison,
       aes(x=Actual, y=Predicted)) +
  geom_point(color="blue", alpha=0.6) +
  labs(title="Actual vs Predicted Values",
       x="Actual Values",
       y="Predicted Values") +
  theme_minimal()
```

OUTPUT:



RESULT:

The scatter plot was successfully generated to visually compare the actual and predicted values of the regression model.

34. Draw a reference line using geom_abline() to compare actual and predicted values.

AIM:

To add a reference line to the actual versus predicted scatter plot using geom_abline().

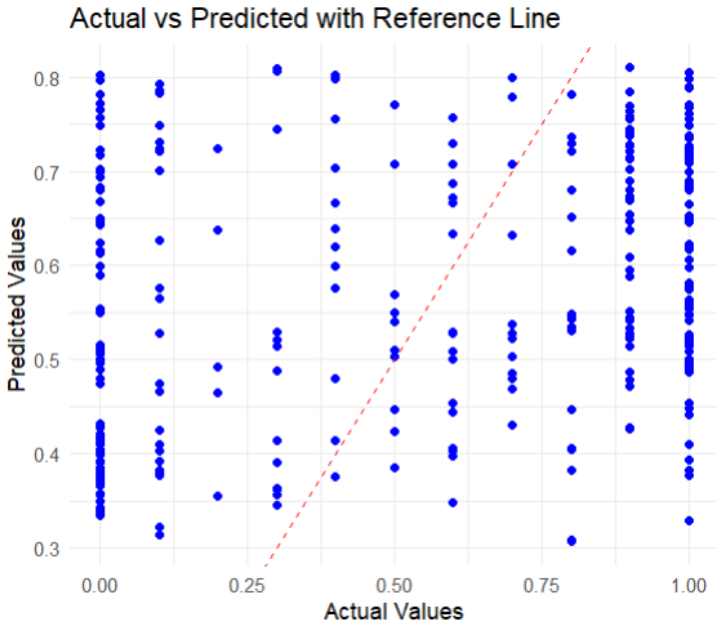
CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
ggplot(comparison,
       aes(x=Actual, y=Predicted)) +
  geom_point(color="blue") +
  geom_abline(intercept=0,
              slope=1,
              color="red",
              linetype="dashed") +
  labs(title="Actual vs Predicted with Reference Line",
       x="Actual Values",
       y="Predicted Values") +
  theme_minimal()
```

OUTPUT:



RESULT:

The reference line was successfully added to compare actual and predicted values. Points closer to the line indicate more accurate predictions.

35. Generate a residual-versus-predicted plot using ggplot2.

AIM:

To visualize the relationship between predicted values and residual errors using a residual-versus-predicted plot.

CODE:

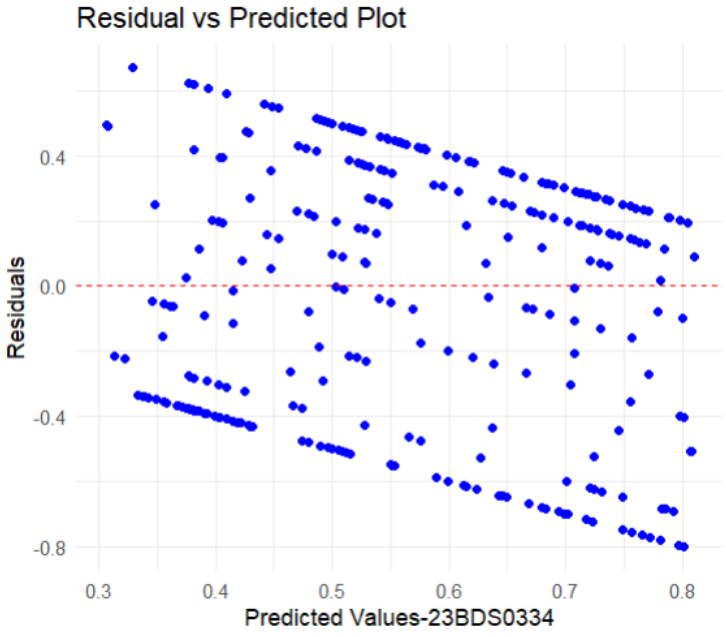
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
residual_data <- data.frame(
  Predicted = predicted,
  Residuals = errors
)

ggplot(residual_data,
       aes(x=Predicted, y=Residuals)) +
  geom_point(color="blue") +
  geom_hline(yintercept=0,
             color="red",
             linetype="dashed") +
  labs(title="Residual vs Predicted Plot",
       x="Predicted Values-23BDS0334",
       y="Residuals") +
  theme_minimal()
```

OUTPUT:



RESULT:

The residual plot was successfully generated to examine the distribution of prediction errors and identify possible patterns in residuals.

36. Plot the distribution of prediction errors using geom_histogram().

AIM:

To visualize the frequency distribution of prediction errors using a histogram.

CODE:

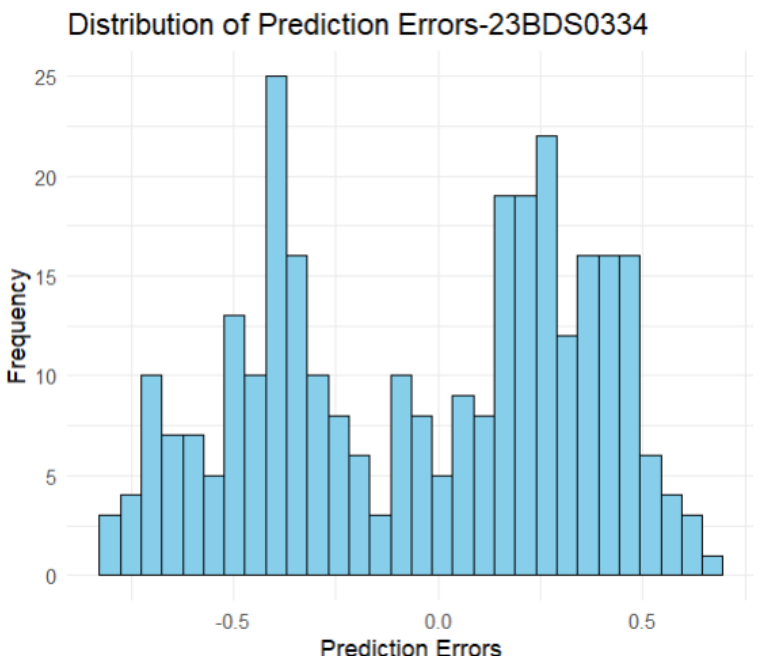
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
error_data <- data.frame(
  Errors = errors
)

ggplot(error_data,
       aes(x=Errors)) +
  geom_histogram(bins=30,
                 fill="skyblue",
                 color="black") +
  labs(title="Distribution of Prediction Errors-23BDS0334",
       x="Prediction Errors",
       y="Frequency") +
  theme_minimal()
```

OUTPUT:



RESULT:

The histogram was successfully generated to understand the spread and distribution of prediction errors.

37. Generate a Q-Q plot of standardized residuals using qqnorm() and qqline().

AIM:

To examine whether standardized residuals follow a normal distribution using a Q-Q plot.

CODE:

assessment_10.R*

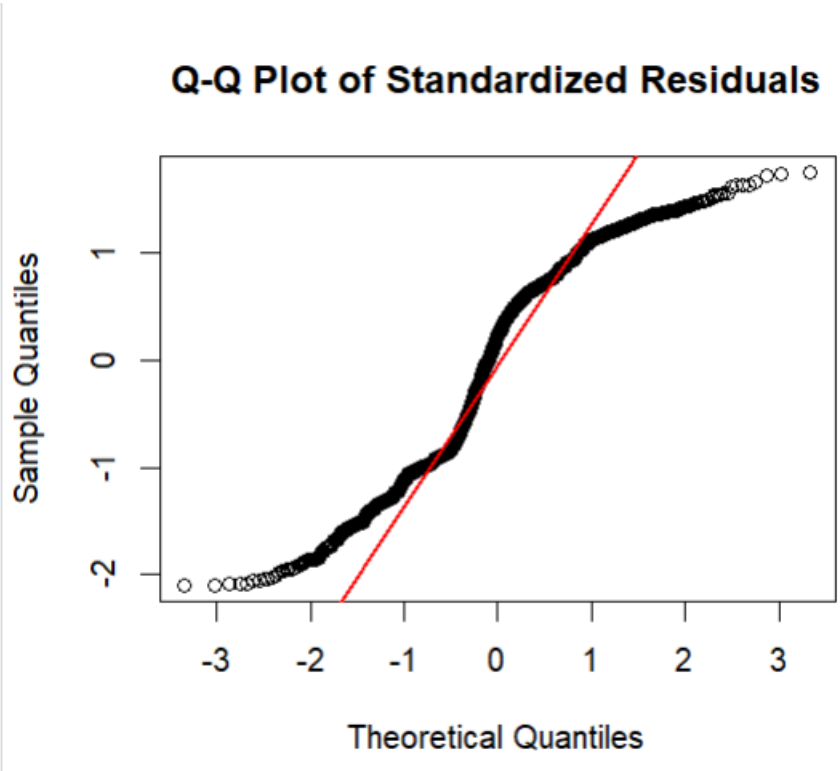
Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Calculate standardized residuals
standardized_residuals <- rstandard(model)

# Generate Q-Q plot
qqnorm(standardized_residuals,
       main="Q-Q Plot of Standardized Residuals")

# Add reference line
qqline(standardized_residuals,
       col="red",
       lwd=2)
```

OUTPUT:



RESULT:

The Q-Q plot was successfully generated to examine the normality of standardized residuals. Points closer to the reference line indicate a closer fit to a normal distribution.

38. Generate a boxplot of prediction errors using geom_boxplot().

AIM:

To visualize the spread, median and potential outliers in prediction errors using a boxplot.

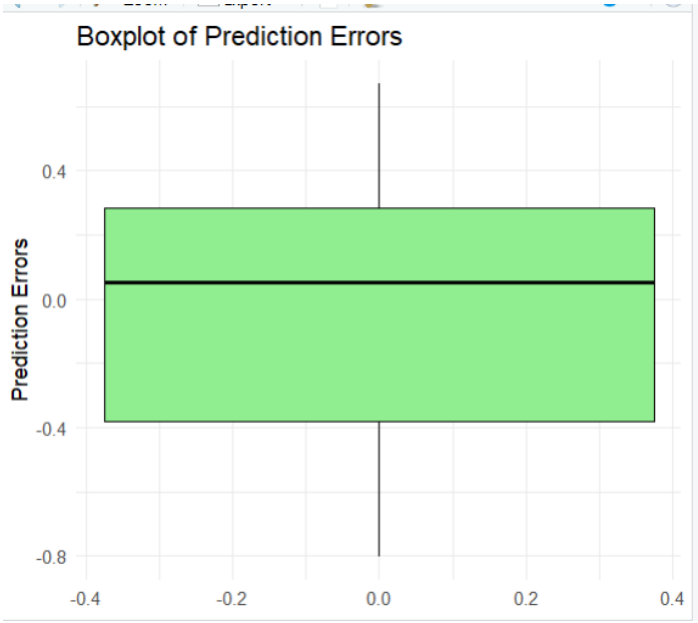
CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
ggplot(error_data,
       aes(y=Errors)) +
  geom_boxplot(fill="lightgreen",
               color="black") +
  labs(title="Boxplot of Prediction Errors",
       y="Prediction Errors") +
  theme_minimal()
```

OUTPUT:



RESULT:

The boxplot was successfully generated to examine the variation and possible extreme values in prediction errors.

39. Visualize the relationship between observation number and prediction error using geom_line().

AIM:

To visualize how prediction errors change across different observations using a line graph.

CODE:

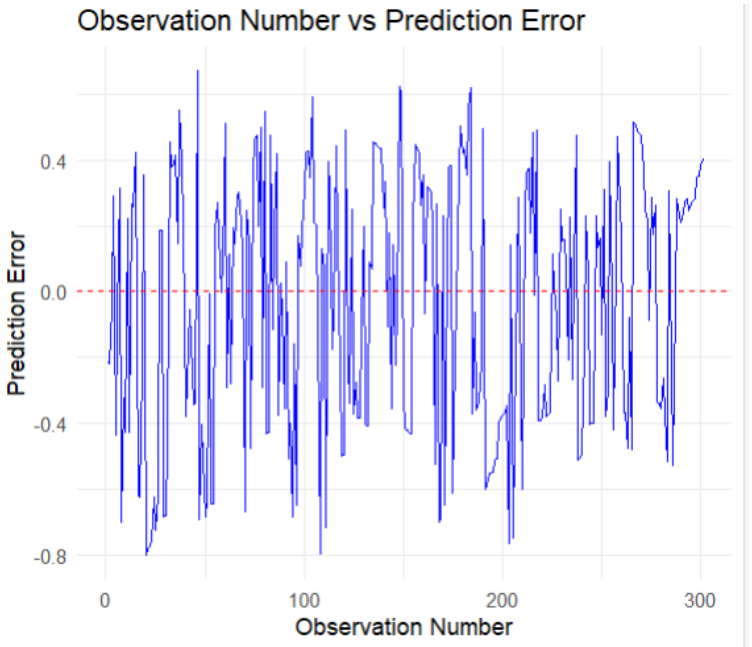
assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
error_data$Observation <- 1:nrow(error_data)

ggplot(error_data,
       aes(x=Observation, y=Errors)) +
  geom_line(color="blue") +
  geom_hline(yintercept=0,
             color="red",
             linetype="dashed") +
  labs(title="Observation Number vs Prediction Error",
       x="Observation Number",
       y="Prediction Error") +
  theme_minimal()
```

OUTPUT:



RESULT:

The line graph was successfully generated to identify changes and patterns in prediction errors across observations.

40. Create a bar chart comparing MAE, MSE, and RMSE using ggplot2.

AIM:

To compare the model evaluation metrics MAE, MSE and RMSE using a bar chart created with ggplot2.

CODE:

assessment_10.R*

Source on Save

```
#23BDS0334
#ARJUNMISHRA
# Calculate evaluation metrics
MAE <- mean(abs(errors))

MSE <- mean(errors^2)

RMSE <- sqrt(MSE)

# Create comparison dataset
metrics <- data.frame(
  Metric = c("MAE", "MSE", "RMSE"),
  Value = c(MAE, MSE, RMSE)
)

print(metrics)

# Generate bar chart
ggplot(metrics,
       aes(x=Metric, y=Value,
           fill=Metric)) +
  geom_bar(stat="identity",
           color="black") +
  labs(title="Comparison of Regression Metrics",
       x="Evaluation Metrics",
       y="Metric Values") +
  theme_minimal() +
  theme(legend.position="none")
```

OUTPUT:

```
> #23BDS0334
> #ARJUNMISHRA
> # Calculate evaluation metrics
> MAE <- mean(abs(errors))
>
> MSE <- mean(errors^2)
>
> RMSE <- sqrt(MSE)
>
> # Create comparison dataset
> metrics <- data.frame(
+   Metric = c("MAE", "MSE", "RMSE"),
+   Value = c(MAE, MSE, RMSE)
+ )
>
> print(metrics)
  Metric     Value
1    MAE 0.3450275
2    MSE 0.1518020
3   RMSE 0.3896177
>
> # Generate bar chart
> ggplot(metrics,
+        aes(x=Metric, y=Value,
+            fill=Metric)) +
+   geom_bar(stat="identity",
+            color="black") +
+   labs(title="Comparison of Regression Metrics",
+        x="Evaluation Metrics",
+        y="Metric Values") +
+   theme_minimal() +
+   theme(legend.position="none")
>
```



RESULT:

The comparison bar chart was successfully generated to visualize the MAE, MSE and RMSE values and understand the prediction errors of the regression model.